

Laporan Praktikum Kontrol Cerdas Week 3

Nama : Cecillia Yowien Arthamevia
NIM : 224308031
Kelas : TKA-6B
Akun Github : <https://github.com/CecilliaYowienArthamevia>
Student Lab Assistant : Salma Halizah Zuhroh

1. Judul Percobaan: *Deep Learning for Intelligent Control Systems*

2. Tujuan Percobaan:

Tujuan dari percobaan deteksi objek dengan *OpenCV* sebagai berikut:

- Memahami konsep dasar *Deep Learning* dalam sistem kendali.
- Mengimplementasikan *Convolutional Neural Network* (CNN) untuk klasifikasi objek.
- Menggunakan *TensorFlow* dan Keras untuk membangun model *Deep Learning*.
- Mengintegrasikan model CNN dengan *Computer Vision* untuk deteksi objek secara *real-time*.
- Menggunakan dataset dari Kaggle untuk pelatihan model.
- Mengembangkan mode *Night Vision* untuk deteksi objek dalam kondisi pencahayaan rendah.

3. Landasan Teori:

Kecerdasan Buatan (*Artificial Intelligence/AI*) merupakan cabang ilmu komputer yang berfokus pada pengembangan sistem yang dapat meniru kecerdasan manusia dalam menyelesaikan masalah. AI mencakup berbagai metode dan teknik, termasuk pemodelan berbasis aturan, pembelajaran mesin (*machine learning*), serta pemrosesan data visual menggunakan teknologi *computer vision* (Indraswara, 2022). AI telah banyak diterapkan dalam berbagai bidang, mulai dari pengolahan data tekstual, suara, hingga citra digital.

Salah satu metode dalam AI yang berkembang pesat adalah *Deep Learning*, yaitu bagian dari *machine learning* yang menggunakan arsitektur jaringan saraf tiruan (*artificial neural networks*) dengan banyak lapisan (*multi-layered neural networks*). *Deep Learning* memungkinkan komputer untuk mengekstrak fitur kompleks dari berbagai bentuk data, seperti gambar dan teks, secara otomatis tanpa perlu ekstraksi fitur manual (Saputra et al., 2023). Model jaringan saraf dalam ini mampu mengenali pola secara efisien melalui proses pembelajaran bertahap yang menyerupai cara otak manusia dalam memproses informasi.

Model yang banyak digunakan dalam *Deep Learning* yaitu *Convolutional Neural Network* (CNN) merupakan arsitektur jaringan saraf tiruan yang dirancang khusus untuk pemrosesan citra digital. CNN bekerja dengan cara mengekstraksi fitur dari gambar melalui operasi konvolusi, yang memungkinkan jaringan untuk mengenali pola visual secara hierarkis, mulai dari fitur sederhana seperti tepi dan sudut hingga fitur yang lebih kompleks seperti bentuk objek (Darmanto et al., 2023). CNN terdiri dari beberapa lapisan utama, yaitu lapisan konvolusi (*convolutional layer*), *pooling layer*, dan *fully connected layer*. Dengan arsitektur ini, CNN telah digunakan dalam berbagai aplikasi seperti klasifikasi gambar, deteksi objek, serta segmentasi citra.

Sementara itu, dalam praktikum kali ini mengintegrasikan model CNN dengan *Computer Vision* untuk deteksi objek secara *real-time*. Dimana *Computer Vision* adalah salah satu penerapan AI yang bertujuan untuk memberikan kemampuan visual kepada komputer, sehingga mesin dapat memahami, menganalisis, dan mengambil keputusan berdasarkan data citra digital atau video. Teknologi ini memanfaatkan algoritma seperti CNN untuk mendeteksi dan mengenali objek dalam gambar, menginterpretasikan gerakan, serta membuat inferensi spasial pada citra 3D (Adhisatria & Wahyudi, 2025).

Dengan kombinasi antara *Computer Vision* dan AI, sistem komputer dapat melakukan tugas-tugas yang sebelumnya hanya dapat dilakukan oleh manusia, seperti pengawasan lalu lintas, analisis medis, dan pengenalan wajah. Secara keseluruhan perkembangan AI dengan menggunakan berbagai model telah membawa revolusi dalam berbagai bidang industri dan penelitian.

Dengan semakin meningkatnya daya komputasi dan ketersediaan data, teknologi ini terus berkembang untuk memberikan solusi yang lebih akurat dan efisien dalam berbagai aplikasi berbasis data visual.

4. Analisis dan Diskusi:

- Analisis

Pada praktikum minggu ketiga ini percobaan yang dilakukan adalah membuat klasifikasi gambar yang terdeteksi dengan model *Deep Learning* menggunakan algoritma *Convolutional Neural Network* (CNN). Tahap pertama yang dilakukan adalah Setup Environment dan Dataset. Selanjutnya, muat dataset dan lakukan *pre-processing* untuk melatih model CNN atau klasifikasi gambar menggunakan Keras atau TensorFlow. Kemudian, kode **ImageDataGenerator** digunakan untuk melakukan augmentasi data pada dataset pelatihan. Augmentasi data adalah teknik untuk memperbanyak variasi data pelatihan dengan melakukan transformasi seperti rotasi, *zoom*, dan *flip* horizontal. Hal ini membantu model menjadi lebih *robust* dan mencegah *overfitting*.

Tahap kedua yang dilakukan adalah membuat dan melatih model CNN untuk klasifikasi objek. Pertama, mendefinisikan model CNN menggunakan Keras Sequential API. Kode **model = Sequential** Ini membuat model sequential, yang berarti lapisan-lapisan dalam model akan dieksekusi secara berurutan. Selanjutnya, mengkompilasi model CNN yang telah kita definisikan sebelumnya menggunakan kode **model.compile**. Kompilasi adalah langkah penting sebelum kita dapat melatih model. Lalu, kode **model.fit** adalah fungsi yang digunakan untuk melatih model. Ini adalah langkah kunci dalam proses pembelajaran mesin, di mana model belajar untuk memetakan input ke output yang diinginkan. Dan terakhir, model yang terlatih disimpan ke file **cnn_model.h5**.

Tahap ketiga yang dilakukan adalah mengintegrasikan dengan OpenCV untuk prediksi *Real-time* dan *Night Vision*. Dengan

menggunakan kamera inisialisasi program untuk mendeteksi objek berdasarkan model CNN yang telah dibuat dengan tambahan fitur Night Vision. Pertama, load model yang telah dilatih menggunakan kode **model = load_model**. Kemudian, load label kelas menggunakan kode **class_labels = list** yang bertujuan untuk mengonversi output numerik dari model CNN (biasanya berupa indeks atau vektor probabilitas) menjadi label kelas yang dapat dibaca manusia.

Selanjutnya, data fitur (nilai RGB) dinormalisasi menggunakan kode **scaler = StandardScaler()**. Normalisasi ini penting untuk memastikan bahwa semua fitur memiliki skala yang sama sehingga dapat meningkatkan kinerja model KNN. Lalu, dilakukan pemisahan dataset menjadi dua bagian yaitu, set pelatihan (*training set*) dan set pengujian (*testing set*) yang digunakan untuk mengevaluasi kinerja model pada data yang tidak dilihat selama pelatihan. Selanjutnya yang dilakukan adalah menginisialisasi kamera menggunakan **cap = cv2.VideoCapture** dengan kamera bawaan PC/Laptop atau dengan *webcam*. Kemudian, menginisialisasi mode *Night Vision* dengan konversi ke skala abu-abu menggunakan kode **night_vision = cv2.cvtColor**. Konversi ke skala abu-abu adalah teknik sederhana namun efektif untuk meningkatkan visibilitas gambar dalam kondisi cahaya redup, yang sering digunakan dalam aplikasi penglihatan malam. Kode **img = cv2.resize** merupakan bagian dari pra-pemrosesan gambar yang sangat penting dalam aplikasi deteksi *objek real-time* menggunakan model CNN. Fungsi **cv2.resize** dari pustaka OpenCV digunakan untuk mengubah ukuran gambar yang diambil dari kamera (frame) menjadi dimensi yang sesuai dengan input model CNN. Kemudian, memprediksi kelas menggunakan kode **model.predict(img)** untuk mengambil gambar yang telah diproses sebelumnya (img) sebagai input dan menghasilkan prediksi. Prediksi ini biasanya berupa array probabilitas, di mana setiap elemen array mewakili probabilitas bahwa gambar tersebut termasuk dalam kelas tertentu. Lalu, mengambil label kelas yang sesuai dari daftar **class_labels**. Dimana **class_labels** adalah daftar yang berisi nama-nama kelas yang sesuai dengan indeks prediksi. Dengan demikian, label akan berisi nama kelas yang

diprediksi oleh model untuk gambar input. Proses ini memungkinkan kita untuk mengubah prediksi numerik dari model menjadi label kelas yang dapat dibaca manusia. Terakhir, menampilkan hasil menggunakan kode **cv2.putText** untuk menambahkan teks ke dalam frame video, kode **cv2.imshow('Frame', frame)** menampilkan frame video asli yang telah ditambahkan teks, dengan judul jendela 'Frame', dan kode **cv2.imshow('Night Vision', night_vision)** untuk menampilkan frame video yang telah diproses untuk fitur penglihatan malam, dengan judul jendela 'Night Vision'. Kedua fungsi **cv2.imshow** ini digunakan untuk melihat hasil pemrosesan video secara *real-time*.

- Diskusi

Dengan kode program mendeteksi objek sesuai dengan pencahayaan tersebut, selanjutnya dilakukan modifikasi program dengan menambahkan *output* secara *real time*, yaitu nilai *Brightness* yang ditampilkan berupa diagram, kondisi pencahayaan, dan waktu dilakukan percobaan mulai dari tanggal, bulan, tahun, jam, dan detik. Nilai outputan tersebut dapat berubah secara *Real Time* yang diakibatkan oleh perubahan cahaya yang terjadi pada objek.

Metode untuk mendeteksi objek sesuai dengan pencahayaan menggunakan metode *Deep Learning* dengan model CNN. *Convolutional Neural Networks* (CNN) adalah algoritma deep learning yang sangat efektif untuk analisis citra, termasuk deteksi objek, karena kemampuannya mengenali pola visual yang kompleks melalui lapisan-lapisan konvolusi, pooling, dan fully connected yang memungkinkan ekstraksi fitur otomatis dari data citra ini memiliki kelebihan dan kekurangan.

Salah satu kelebihan utama CNN adalah kemampuannya dalam mengidentifikasi fitur-fitur rendah seperti tepi, tekstur, atau warna dari gambar, bahkan dalam kondisi pencahayaan yang berbeda. Lapisan konvolusi pada CNN bertanggung jawab untuk mendeteksi karakteristik tertentu seperti bentuk objek atau posisi relatifnya. Selain itu, CNN juga

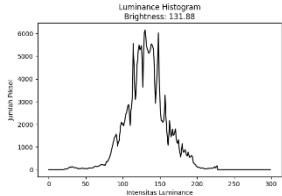
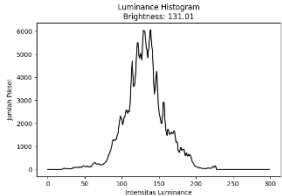
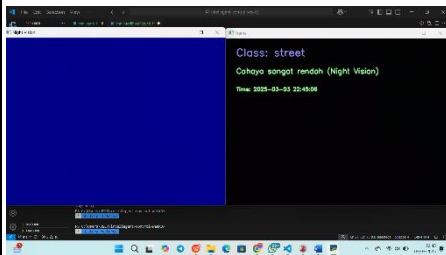
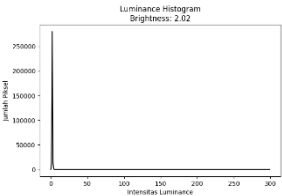
mampu mendeteksi objek dengan sudut tubuh serta posisi pencahayaan yang berbeda.

Namun, CNN juga memiliki keterbatasan. Algoritma CNN memerlukan pencahayaan yang konsisten untuk performa optimal. CNN mengalami kesulitan ketika digunakan untuk mendeteksi objek di malam hari. Untuk mengatasi masalah ini, dapat digunakan metode *Gamma Correction* untuk meningkatkan kualitas gambar, sehingga YOLO (*You Only Look Once*) dapat dengan mudah mengenali objek pada gambar yang gelap dan kurang pencahayaan. Metode *Gamma Correction* dapat menambahkan kontras dan warna terhadap suatu gambar yang gelap agar objek yang tidak terkena cahaya sama sekali dapat terlihat dengan jelas.

5. Assignment :

Dalam praktikum minggu ketiga ini yang dilakukan adalah memodifikasi kode program dengan penambahan fitur sehingga dapat menampilkan output nilai *Brightness*, keterangan kondisi pencahayaan pada objek, dan keterangan waktu dilakukannya percobaan, dimana data tersebut muncul secara *Real Time*. Dimulai dengan menganalisis spektrum cahaya berdasarkan luminance. Selanjutnya mengkonversi ke YUV untuk mendapatkan luminance. Kemudian menampilkan histogram dengan menggunakan kode program `plt.figure(figsize=(7, 5))`. Selanjutnya menampilkan label pada histogram dengan kode program `plt.plot(hist_y, color='black'), plt.title(f'Luminance Histogram\nBrightness: {avg_luminance:.2f}'), plt.xlabel('Intensitas Luminance'), plt.ylabel('Jumlah Pixel'), plt.subplots_adjust(bottom=0.2)`. Lalu menampilkan output waktu dengan kode program `current_time = datetime.datetime.now().strftime("%d-%m-%Y %H:%M:%S")`. Kemudian dilanjutkan dengan menentukan kategori pencahayaan berdasarkan luminance yang mana terdapat keadaan cahaya sangat rendah, cahaya sedang, dan cahaya terang. Kemudian menganalisis spektrum cahaya dengan kode program `light_status = analyze_light_spectrum(frame)`, dan akan ditampilkan di frame output secara *Real Time*.

6. Data dan Output Hasil Pengamatan:

No	Kondisi	Hasil Percobaan	Output
1.	Pada kondisi siang hari di ruangan dengan keadaan lampu menyala		
2.	Pada kondisi sore hari di ruangan		
3.	Pada kondisi malam hari di jalan dengan keadaan lampu mati		

7. Kesimpulan:

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- Metode yang digunakan dalam praktikum ini adalah untuk mendeteksi objek dengan dipengaruhi oleh cahaya menggunakan metode *Deep Learning* dengan menggunakan software *Visual Studio Code*.
- Metode *Convolutional Neural Network* (CNN) yang diimplementasikan dengan deep learning memiliki kemampuan dalam pengenalan objek, termasuk dalam kondisi pencahayaan yang kurang atau pada malam hari.
- Algoritma Metode *Convolutional Neural Network* CNN memerlukan pencahayaan yang konsisten untuk performa optimal.
- Semakin besar cahaya yang terdeteksi pada objek maka nilai *Brightness* yang dihasilkan akan semakin besar.
- Keakurasian hasil data percobaan dapat dipengaruhi oleh dataset yang digunakan dan spesifikasi kamera laptop atau PC.

8. Saran:

Untuk mengatasi tantangan dan kekurangan pada metode *Deep Learning* dengan CNN (*Convolutional Neural Network*), terdapat beberapa saran yang dapat digunakan salah satunya yaitu dengan meningkatkan kualitas dan kuantitas dataset pelatihan, terutama yang mencakup variasi kondisi pencahayaan ekstrem dan noise. Teknik augmentasi data dapat digunakan untuk memperluas dataset secara sintesis, sehingga model menjadi lebih robust terhadap variasi kondisi lingkungan. Selanjutnya dapat dilakukan eksplorasi dan pengembangan arsitektur CNN yang lebih adaptif terhadap perubahan pencahayaan, misalnya dengan mengintegrasikan lapisan normalisasi adaptif atau mekanisme perhatian. Kemudian penggunaan teknik pra-pemrosesan gambar seperti peningkatan kontras, penghilangan noise, dan normalisasi pencahayaan dapat membantu meningkatkan kualitas input bagi CNN. Dan penerapan metode *Gamma Correction* untuk meningkatkan kualitas gambar dalam kondisi minim cahaya, sehingga membantu algoritma seperti YOLOv5 untuk mengenali objek dengan lebih baik.

9. Daftar Pustaka:

- Adhisatria, M. I. B., & No, J. S. (2025). Perancangan Kontrol Adaptif Kepadatan Arus Lalu Lintas dengan Metode Convolutional Neural Network. 8.
- Darmanto, B. F., Setianingsih, C., & Saputra, R. E. (n.d.). Deteksi Pelanggaran pada Bahu Jalan Tol Menggunakan Algoritma Cascade R-CNN.
- Oleh, D., & Indraswara, A. J. (2022). PERANCANGAN SISTEM MACHINE LEARNING UNTUK KLASIFIKASI KENDARAAN PADA PERSIMPANGAN TRAFFIC LIGHT.
- Setiawan, I. (2022). MENGATASI KEMACETAN DI LAMPU MERAH DENGAN PENDEKATAN IMAGE PROCESSING. *Journal of Innovation And Future Technology (IFTECH)*, 4(2), 9–18. <https://doi.org/10.47080/iftech.v4i2.2117>.
- Saputra, V. P., Latifa, U., & Ibrahim, I. (2023). Simulasi detection counter pada objek kendaraan motor dan mobil menggunakan metode Convolutional Neural Network berbasis Python.